

1 Excepciones Personalizadas

1.1 Jerarquía de excepciones

Todas las excepciones del dominio extienden de **ClinicaException**, que a su vez extiende **RuntimeException**. Esto las convierte en excepciones *unchecked*: no obligan a declarar throws en la firma del método, pero pueden capturarse con un único catch (ClinicaException e) en la capa de presentación. Cada excepción incluye un **código de error** textual que permite identificarla programáticamente.

RuntimeException

```

■■■ ClinicaException // base del proyecto, con código de error
    ■■■ PacienteNoEncontradoException // PACIENTE_NO_ENCONTRADO
    ■■■ OdontologoNoEncontradoException // ODONTOLOGO_NO_ENCONTRADO
    ■■■ TurnoNoEncontradoException // TURNO_NO_ENCONTRADO
    ■■■ TurnoYaReservadoException // TURNO_YA_RESERVADO
    ■■■ MatriculaDuplicadaException // MATRICULA_DUPLICADA
    ■■■ DniDuplicadoException // DNI_DUPLICADO
    ■■■ OperacionNoPermitidaException // OPERACION_NO_PERMITIDA
    ■■■ DatoInvalidoException // DATO_INVALIDO
        ■■■ FechaInvalidaException // hereda DATO_INVALIDO
  
```

1.2 Descripción y cuándo se lanza cada excepción

Excepción	Cuándo se lanza	Capa
PacienteNoEncontradoException	Al buscar un paciente por ID o DNI que no existe en el repositorio.	Servicio
OdontologoNoEncontradoException	Al buscar un odontólogo por ID o matrícula que no existe.	Servicio
TurnoNoEncontradoException	Al buscar un turno por ID que no existe en el repositorio.	Servicio
TurnoYaReservadoException	Al reservar un turno con un odontólogo que ya tiene ese horario ocupado (Agenda.estaDisponible() retorna false).	Servicio
MatriculaDuplicadaException	Al registrar un odontólogo con una matrícula que ya existe en el sistema.	Servicio
DniDuplicadoException	Al registrar un paciente con un DNI que ya existe en el sistema.	Servicio
OperacionNoPermitidaException	Al intentar eliminar un paciente u odontólogo con turnos futuros asignados.	Controlador

Excepción	Cuándo se lanza	Capa
DatoInvalidoException	Al ingresar datos inválidos: campo vacío, rango de fechas erróneo, formato incorrecto.	Servicio
FechaInvalidaException	Al intentar reservar o modificar un turno con una fecha anterior a la actual.	Servicio

1.3 Clase base ClinicaException

La clase base define el contrato común: un constructor que acepta **codigo** y **mensaje**, un getter `getCodigo()` y un `toString()` que formatea `[CODIGO] mensaje` para facilitar el log y los mensajes de consola.

```
public class ClinicaException extends RuntimeException {  
  
    private final String codigo;  
  
    public ClinicaException(String codigo, String mensaje) {  
        super(mensaje);  
        this.codigo = codigo;  
    }  
  
    public String getCodigo() { return codigo; }  
  
    @Override  
    public String toString() {  
        return "[" + codigo + "] " + getMessage();  
    }  
}
```

2 Manejo Robusto de Errores

2.1 Estrategia por capas

Las excepciones se lanzan en la capa más cercana al dominio y se propagan hacia arriba hasta ser capturadas en la presentación, donde se muestran mensajes amigables al usuario. La tabla siguiente resume la responsabilidad de cada capa:

Capa	Responsabilidad de manejo de errores
Modelo	La lógica de negocio intrínseca (disponibilidad de agenda, turno futuro) genera estados que el servicio detecta y convierte en excepciones.
Servicio	Valida precondiciones y lanza <code>ClinicaException</code> con mensajes descriptivos. No captura excepciones propias; las propaga hacia arriba.
Controlador	Coordina múltiples servicios y lanza <code>OperacionNoPermitidaException</code> cuando la composición de reglas lo exige (p. ej., eliminar con turnos futuros).
Presentación	Captura <code>ClinicaException</code> con un único <code>catch</code> y muestra el mensaje. Los bucles de entrada re-solicitan datos inválidos sin terminar el flujo.

2.2 try-catch-finally en operaciones de IO

PersistenciaCSV utiliza dos estilos de manejo. La mayoría de métodos usa *try-with-resources* para cierre automático del stream. El método **cargarPacientes()** usa *try-catch-finally* explícito para demostrar el control manual del ciclo de vida del reader, garantizando el cierre incluso ante excepciones inesperadas:

```
// try-catch-finally explícito — cargarPacientes()
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader(PACIENTES_CSV));
    // ... procesamiento línea a línea ...
} catch (IOException e) {
    throw e; // propaga al llamador para que decida
} finally {
    if (br != null) {
        try { br.close(); } catch (IOException ignored) {}
    }
}
```

2.3 Validación en la capa de presentación

ConsolaUtils centraliza todos los métodos de entrada con validación y reintento automático:

- **leerTexto()** — rechaza cadenas vacías o con solo espacios.
- **leerEmail()** — verifica que el texto contenga '@'.
- **leerFechaNoPasada()** — re-solicita si la fecha es anterior a hoy.

- **leerIdExistente()** — acepta un Consumer; si lanza excepción, repite el ciclo.

3

Colecciones Avanzadas

Los servicios utilizan **Stream API** de forma extensiva para filtrar, ordenar y transformar colecciones sin modificar el repositorio. Se implementan tanto **Comparable** como **Comparator** según el contexto.

3.1 Operaciones implementadas

Operación	Implementación
Pacientes ordenados alfabéticamente	Paciente implementa Comparable comparando apellido y luego nombre. Collections.sort() y stream().sorted() lo aprovechan.
Odontólogos ordenados	stream().sorted(Comparator.comparing(Odontologo::getApellido).thenComparing(Odontologo::getNombre))
Turnos ordenados por fecha/hora	Turno implementa Comparable; Collections.sort() en listarTodos().
Filtrar turnos por paciente	stream().filter(t -> t.getPaciente().getId().equals(id)).sorted().collect(Collectors.toList())
Filtrar turnos por odontólogo	stream().filter(t -> t.getOdontologo().getId().equals(id)).sorted().collect(Collectors.toList())
Filtrar por rango de fechas	stream().filter(t -> !t.getFecha().isBefore(desde) && !t.getFecha().isAfter(hasta)).sorted()
Filtrar por estado	stream().filter(t -> t.getEstado() == estado).sorted().collect(Collectors.toList())
Carga CSV con relación ID→objeto	HashMap para acceso O(1); Iterator explícito sobre odontólogos para construir el mapa.

3.2 Persistencia en archivos CSV

La clase **PersistenciaCSV** implementa lectura y escritura con **BufferedReader** / **BufferedWriter**. Los datos se organizan en tres archivos con separador ';' para evitar conflictos con comas en los datos:

Archivo	Columnas
pacientes.csv	id; nombre; apellido; email; dni; fechaIngreso; domId; calle; numero; localidad; provincia
odontologos.csv	id; nombre; apellido; email; matricula
turnos.csv	id; pacientId; odontologoId; fecha; hora; estado; recepcionistaLegajo

La carga ocurre automáticamente al iniciar la aplicación y el guardado se dispara en cada operación CRUD, garantizando que los datos persisten sin intervención del usuario.

```
// Auto-persistencia en cada operación – RepositorioTurno
@Override
public void guardar(Turno turno) {
    almacenamiento.add(turno);
    persistir(); // escribe todo el CSV inmediatamente
}

private void persistir() {
    try { PersistenciaCSV.guardarTurnos(buscarTodos()); }
    catch (IOException e) { System.err.println("■ " + e.getMessage()); }
}
```

4

Diagramas de Secuencia

4.1 Alta de un nuevo turno

El diagrama muestra el flujo completo de reserva: validación de IDs en bucle, validación de fecha, y las dos ramas de excepción posibles — entidad no encontrada y horario ya ocupado. Los participantes cubren todas las capas: UI → Controlador → Servicio → Repositorio.

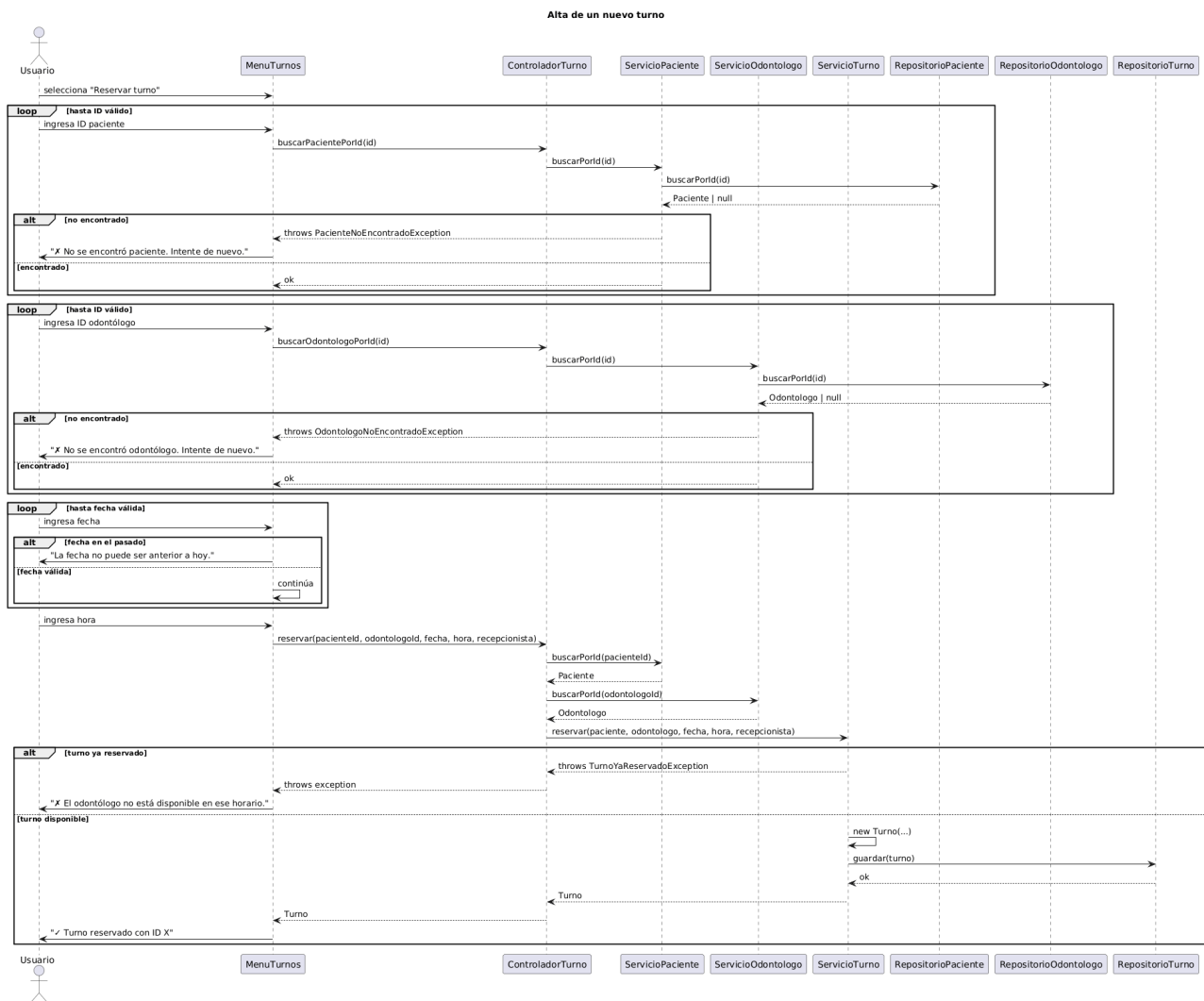


Figura 1 — Diagrama de secuencia: Alta de un nuevo turno

4.2 Búsqueda de paciente por DNI

Muestra el caso exitoso (paciente encontrado, se muestran sus datos) y el flujo alternativo (**PacienteNoEncontradoException**). También incluye la validación de campo vacío con **DatoInvalidoException** antes de consultar el repositorio.

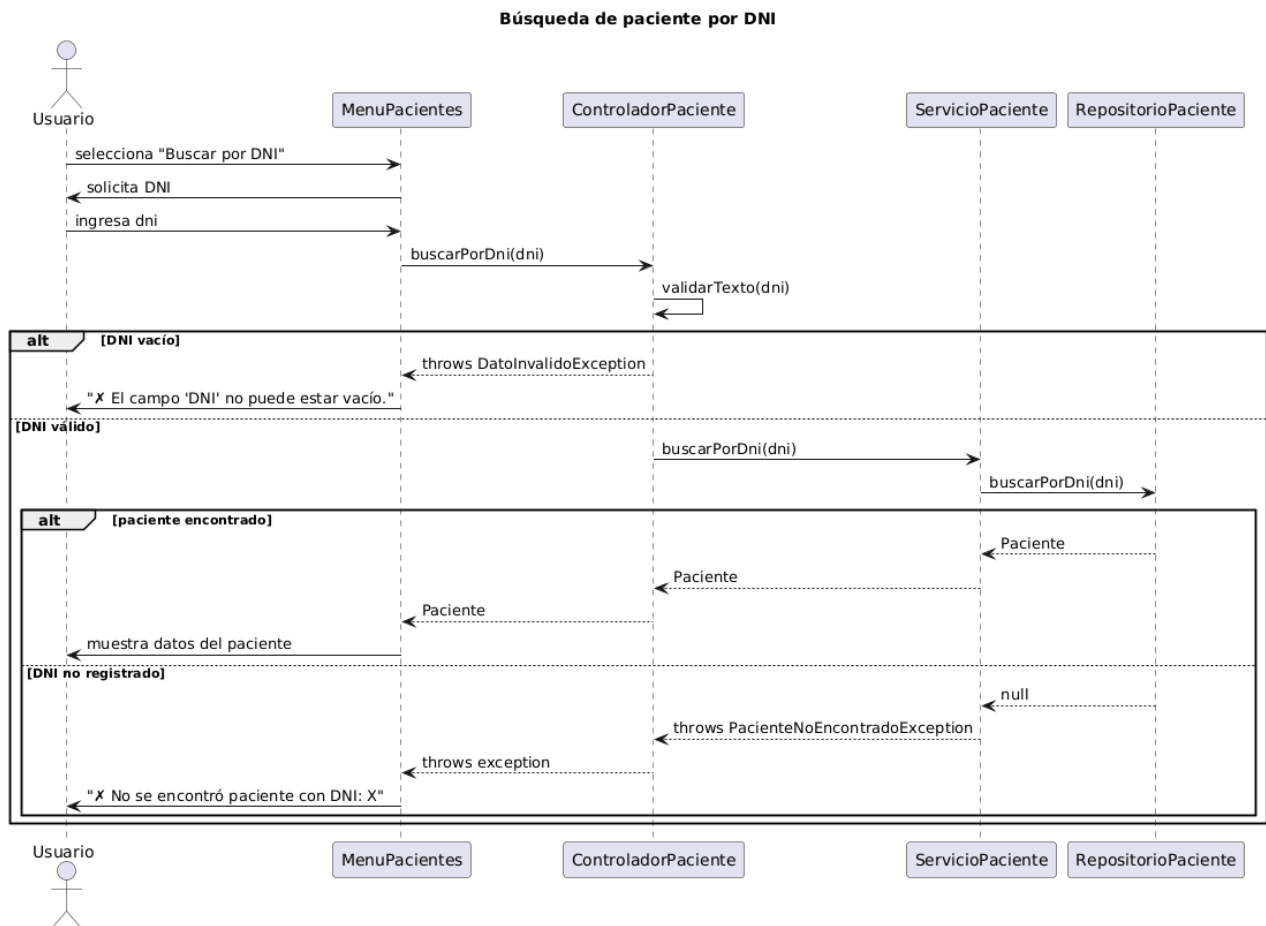


Figura 2 — Diagrama de secuencia: Búsqueda de paciente por DNI

5

Conclusiones

La Entrega 3 consolida el sistema con tres pilares clave:

- **Robustez:** la jerarquía de excepciones con clase base **ClinicaException** garantiza que ningún error pase desapercibido. Cada excepción tiene un código único y un mensaje descriptivo, y la capa de presentación nunca colapsa ante un error.
- **Expresividad:** la Stream API reemplaza bucles imperativos por pipelines declarativos legibles. La implementación de **Comparable** y **Comparator** asegura ordenamientos correctos y reutilizables en cualquier contexto.
- **Persistencia transparente:** el patrón de auto-guardado en cada operación CRUD libera a la UI de gestionar explícitamente el ciclo de vida de los datos. La carga automática al inicio hace que el sistema sea stateful entre ejecuciones sin fricciones para el usuario.

Con estas tres mejoras el sistema cumple todos los requisitos de la Entrega 3 y sienta las bases para la Entrega 4 (interfaz gráfica Swing), donde la robustez del modelo de errores y la persistencia ya probada serán reutilizadas sin cambios.